Runtime Enforcement of Safe Lane-Change Behavior in Autonomous Vehicles using DFA-based Automata Learning

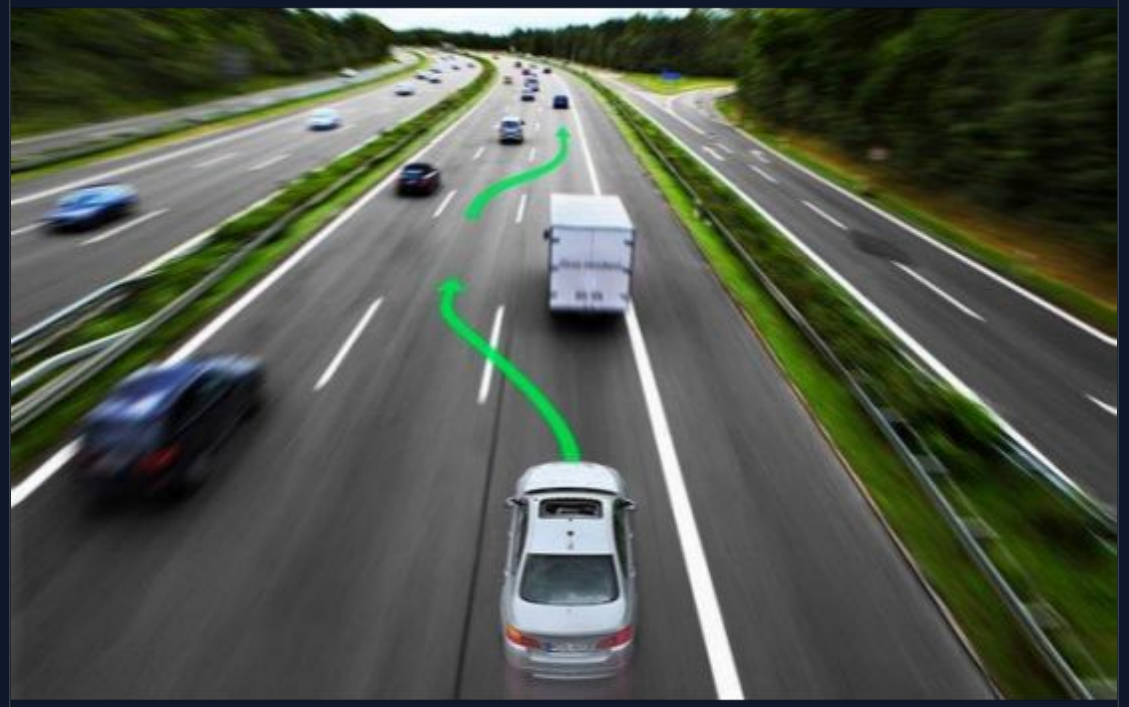
Nirbhay Sharma Harsh Gupta

Supervisor: Saumya Shankar| Duration: 16–18 Weeks

Motivation

- Autonomous vehicles (AVs) depend on machine-learned control policies for decision-making.
- Lane-change is one of the most safety-critical maneuvers: requires signalling, maintaining safe distance, and smooth merging.
- Learned or heuristic controllers can produce unsafe sequences due to:
  - Perception errors (noisy sensors)
  - Planning faults (unsafe gap estimation)
  - Environment unpredictability (cut-ins)
- Offline verification cannot capture every possible condition.

→**Therefore, runtime enforcement is needed to continuously monitor the AV's decisions and block unsafe behaviors.**



Problem Statement

Problem

Even advanced autonomous driving stacks may issue unsafe lane-change commands (e.g., lane shift without turn signal or insufficient gap).

Goal

Build a runtime enforcer that monitors event sequences and ensures every lane-change decision complies with a formally learned model of safety.

Method

Use Deterministic Finite Automata (DFA) learned from safe driving traces to define allowed event sequences and block non-compliant commands.

Objectives

- Collect safe and unsafe lane-change traces from an AV simulator.
- Learn a DFA that represents safe lane-change behavior patterns.
- Define safety rules as regular languages over event sequences.
- Implement a runtime enforcer (shield) that checks every event transition against the DFA.
- Formally verify safety and minimal interference properties.
- Evaluate performance and safety in simulation.

Core Research Questions

- How can safe lane-change behavior be modeled as a regular language?
- Can a learned DFA generalize across varied traffic and environmental conditions?
- How can we guarantee that the runtime enforcer's allowed sequences are always a subset of safe sequences?
- What is the trade-off between safety enforcement and controller flexibility (minimal interference)?

Conceptual Overview

AV Stack (Controller)

Generates event stream: {signal_on, gap_ok, lane_change_start, ...}

→

Runtime Enforcer (Shield)

Tracks DFA state. Checks if next event is allowed. If unsafe → block/correct.

→

Safe Control Commands

Verified, safe commands passed to the vehicle's actuators.

Automata Learning Framework

Alphabet (Σ) & Traces

Alphabet: { signal_on, safe_gap, unsafe_gap, lane_change, stabilize, abort }

Positive Traces (Safe):
signal_on → safe_gap → lane_change → stabilize

Negative Traces (Unsafe):
lane_change → unsafe_gap → stabilize
signal_on → lane_change (no safe_gap)

Learning Algorithm

Algorithm:
RPNI (Regular Positive & Negative Inference) or L* Algorithm (Angluin, 1987).

Tool:
AALpy (Python).

Goal:
Infer a minimal DFA that accepts all safe traces and rejects all unsafe ones.

Runtime Enforcement Design

Mechanism

Input: Event sequence stream from the AV planner (via CARLA API).

Logic: Maintain current DFA state. On each new event *e*:

- If DFA has a valid transition from *current_state* with *e*, → update state.
- Else → block command, trigger corrective action.

Output: Stream of verified safe commands to the control layer.

Enforcement Property

Let Lsafety = language of safe DFA, and Lenforcer = allowed sequences by the shield.

Then:

$$L_{enforcer} \subseteq L_{safety}$$

✔ **Ensures no unsafe sequence is ever allowed.**

Formal Verification

Safety Guarantee

Prove that unsafe DFA states (rejecting states) are unreachable when the enforcer is active.

Minimal Interference

If the controller's output is already in Lsafety, the enforcer does not alter it.

Verification Tools

PyNuSMV, Language inclusion checking, and proofs of determinism/completeness.

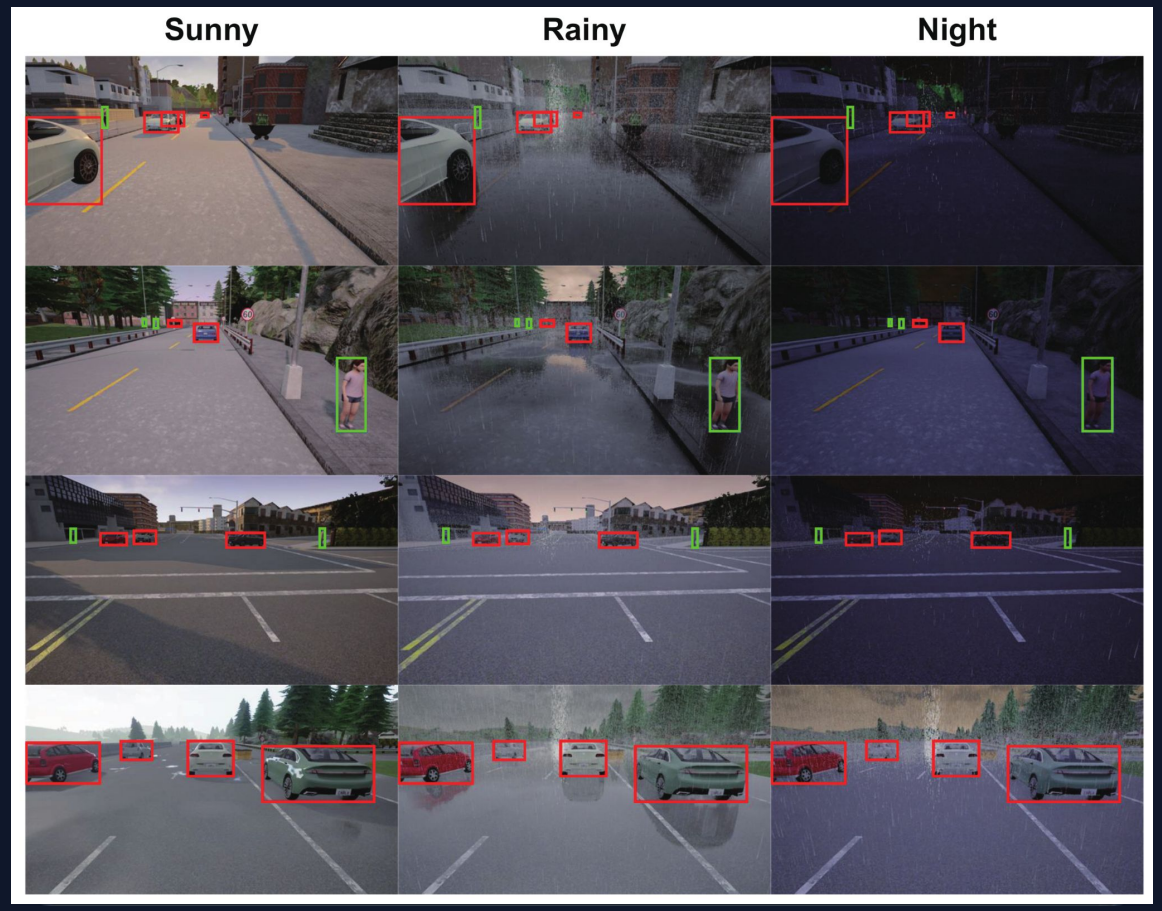
Simulation Environment

Tools & Setup

- **Simulator:** CARLA 0.9.14+ (Python API)
- **Scenarios:** Lightweight, realistic lane-change scenarios on Town03 / Town04.
- **Hardware:** Use off-screen rendering for speed (no GPU bottleneck). 2060 Super GPU sufficient.

Data Collection

- Collect traces of turn signal, gap detection, lane offset, velocity, and acceleration.
- **Dataset:** 50–100 lane-change traces (safe & unsafe), each ≈ 10–15 events.



Evaluation Metrics

Metric	Description	Target
Safety Violation Rate	% of unsafe sequences executed	0%
False Positive Rate	% of safe commands wrongly blocked	< 10%
Latency Overhead	Avg delay per event check	< 100 ms
Generalization Accuracy	DFA performance on unseen scenarios	> 90%
Minimal Interference	Ratio of blocked unsafe / total interventions	> 95% meaningful

Risks and Mitigation

Risk	Mitigation
Limited variety of traces → poor DFA generalization	Use parameterized scenario generation (vary traffic density, gap thresholds)
DFA overfitting	Use abstraction (coarser event alphabet) + RPNI regularization
Integration lag / performance bottleneck	Enforcer checks at decision-level granularity, not every frame
Non-deterministic event order	Design alphabet carefully for deterministic sequence extraction

Expected Contributions

- A data-driven DFA model capturing safe lane-change behavior.
- A runtime enforcer implementing online safety validation for AV planners.
- Formal proofs of safety and minimal interference properties.
- Empirical results demonstrating zero unsafe maneuvers under enforcement.
- Framework extendable to other maneuvers (merging, overtaking).

Future Work

- Extend DFA to Probabilistic DFA (PDFA) to handle stochastic traffic conditions.
- Use online learning to adapt DFA during runtime from new data.
- Integrate multiple DFAs for composite safety (speed, intersection, pedestrian).
- Explore integration with reinforcement learning agents for runtime safety shields.